Alize Programming Language Design (Version 0.1)

Tokens

- **Type Literals**
  - Undefined
  - Byte // 'b'
  - Char // 'c'
  - Integer // 'i'
  - Float // 'f'
  - String // 's'
  - Tuple
  - Array // 'a'
  - HashSet
  - HashMap
- **Constant Literals**
  - true
  - false
  - undefined
  - IDENTIFIER
- **Reserved Keyword Literals**
  - struct
  - fun
  - lambda
  - macro
  - let
  - var
  - @define
  - @operator
  - @overload
  - @properties
  - @initialize
  - @static
  - @fun
  - @impl
  - @testing

  - @test
  - @setup
  - @cleanup
  - import
  - from
  - alias
  - self
  - trait
  - implements
  - if
  - for
  - loop
  - else
  - elif
  - when
  - match
  - case
  - with
  - break
  - continue
  - return
  - Ok
  - Err
  - Some
  - None
  - delete
  - @skip

- **Other tokens** "-", "+", "*", "/", "%", ",", "=", "==", "!=", "<=", ">=", "<", ">", "«", "»", "|", "&", "~", "^", "'", ".", ":", "::", "{", "}", "(", ")", "[", "]", "$", "$!", "$%", "and", "or", "not", "delete",
- **Overrideable operators** "-", "+", "*", "/", "%", "==", "!=", "<=", ">=", "<", ">", "«", "»", "|", "&", "~", "^"
- **Overrideable methods** "___str___", // format("{name:s}") | obj.str() | String(obj) "___repr___", // format("{name:r}") | obj.repr() | repr(obj) "___and___", // lhs and rhs "___or___", // lhs or rhs "___not___", // not rhs "___hash___" // hash(obj) "___delete___", // delete ident "___delitem___" // delete ident[idx] "___setprop___", // obj.member = expr "___getprop___", // obj.member "___setitem___", // obj[expr] = expr "___getitem___", // obj[expr] "___next___", // obj = new Iteraor(Self), next(obj) "___has_next___", // Some(val) = next(obj) | None = next(obj)

Core Types

code-block:: c++

    class Arg; class Object; class Iterable; class Error; class Env; class Parameter; class ParameterEqual; class ParameterHash; class Matcher; class Identifier; class IdentifierEqual; class IdentifierHash; class Property; class PropertyEqual; class PropertyHash; class Signature; class SignatureEqual; class SignatureHash; class SelfEqual; class SelfHash; class Module; class ModuleEqual; class ModuleHash; class Method; class MethodEqual; class MethodHash; class Symbol; class SymbolEqual; class SymbolHash; class Collection; class Sequence; class Mapping;

    using u8 = std::uint8_t; using i8 = std::int8_t; using u16 = std::uint16_t; using i16 = std::int16_t; using u32 = std::uint16_t; using i32 = std::int32_t; using u64 = std::uint64_t; using i64 = std::int64_t; using f32 = float; using f64 = double; using Str = std::string; using usize = std::uint64_t; using isize = std::int64_t; using StrIterator = typename Str::iterator; using Any = std::any; template < typename T> using Vec = std::vector<T>; template < typename T> using SharedPtr = std::shared_ptr<T>; template < typename T> using UniquePtr = std::unique_ptr<T>; template < typename KeyT, typename ValT,typename EqualT=?,typename HashT=??> using Dict = std::unordered_map<KeyT, ValT, EqualT, HashT>; template < typename KeyT, typename ValT,typename EqualT=?,typename HashT=??> using Set = std::unordered_set<KeyT, ValT, EqualT, HashT>;

    using Self = SharedPtr<Object>; using Args = Vec<Arg>; using CFun = Self (*)(const Args&); using CMethod = Self (Object::*)(const Vec<Arg>& args); using ValueList = std::list<Self>; using Iterator = SharedPtr<Iterable>; using Optional = std::optional<Self>;

code-block:: c++

    class Object; class Undefined; class Bool; class Byte; class Char; class Number; class String; class Tuple; class Array; class BytesArray; class ArrayList; class List; class HashSet; class HashMap; class Callable; class Iterable; class StringIterator; class TupleIterator; class ArrayIterator; class BytesArrayIterator; class ArrayListIterator class ListIterator; class Range; class Linspace; class HashSetIterator; class HashMapIterator; class Structure; class Trait; class Option; class Result;

Base Object

code-block:: c++

class Object{ /* #m_typename: Identifier; #m_properties: Self; #m_methods: Dict<Identifier, Self, IdentifierEqual, IdentifierHash>; #m_impls: Dict<Identifier, Self, IdentifierEqual, IdentifierHash>; #m_operators: Dict<Identifier, Self, IdentifierEqual, IdentifierHash>; #m_cmethods: Dict<Str, CMethod, std::hash<Str>, std::equal<Str»; +virtual ~Object(); +explicit Object() noexcept; +explicit Object(Str tyname) noexcept; +virtual Symbol type(void) const; +size_of(void) -> usize;

// -- *Builtin predicates* --virtual bool is_object(void) const; virtual bool is_undefined(void) const; virtual bool is_bool(void) const; virtual bool is_char(void) const; virtual bool is_byte(void) const, virtual bool is_integer(void) const; virtual bool is_float(void) const; virtual bool is_complex(void) const; virtual bool is_number(void) const; virtual bool is_string(void) const; virtual bool is_tuple(void) const; virtual bool is_array(void) const; virtual bool is_arraylist(void) const; virtual bool is_list(void) const; virtual bool is_hashset(void) const; virtual bool is_hashmap(void) const; virtual bool is_bytearray(void) const; virtual bool is_cfunction(void) const; virtual bool is_lambda(void) const; virtual bool is_function(void) const; virtual bool is_macro(void) const; virtual bool is_callable(void) const; virtual bool is_iterable(void) const; virtual bool is_string_iterator(void) const; virtual bool is_array_iterator(void) const; virtual bool is_tuple_iterator(void) const; virtual bool is_arraylist_iterator(void) const; virtual bool is_list_iterator(void) const; virtual bool is_hashset_iterator(void) const; virtual bool is_hashmap_iterator(void) const; virtual bool is_newtype(void) const; virtual bool is_optional(void) const; virtual bool is_ok(void) const; virtual bool is_error(void) const; virtual bool is_some(void) const; virtual bool is_none(void) const; ... virtual is_matchable(void) -> bool virtual is_hashable(void) -> bool virtual is_cloneable(void) -> bool virtual is_moveable(void) -> bool virtual is_formattable(void) -> bool // -- *Builtin type-cast methods* --virtual as(Symbol) -> Self +virtual from(other: Self) -> Self +virtual clone(void) -> Self +virtual swap(other: Self) -> void +virtual match(other: self) -> bool +hasProperty(propname: Identifier)-> bool +defineProperty(propname: Identifier, propval: Self) -> void +getProperty(propname: Identifier) -> Self; +defineOperator(opname: Str, Self&& fun) -> void +implementTrait(name: Identifier, SharedPtr<Trait>) -> void +virutal iter(void) const -> Iterator; +virtual str(void) const -> Str; +virtual repr(void) const -> Str; +virtual call(const Str& name, const Vec<Self>&

args) -> Self; */

};

Fundamental Types

code-block:: c++

class Undefined final : public Object {}; class Bool final : public Object{ /* -m_data: bool; Bool& operator!(); friend bool operator==(const Bool& lhs, const Bool& rhs); friend bool operator!=(const Bool& lhs, const Bool& rhs); friend bool operator||(const Bool& lhs, const Bool& rhs); friend bool operator&&(const Bool& lhs, const Bool& rhs); */ }; class Byte final : public Object{ /* -m_data: u8; Byte& operator~(); type(void) const override -> Symbol; is_byte(void) const override -> bool; friend bool operator==(const Byte& lhs, const Byte& rhs); friend bool operator!=(const Byte& lhs, const Byte& rhs); friend bool operator«(const Byte& lhs, const Byte& rhs); friend bool operator»(const Byte& lhs, const Byte& rhs); friend bool operator«(const Byte& lhs, const Byte& rhs); friend bool operatorˆ(const Byte& lhs, const Byte& rhs); friend bool operator|(const Byte& lhs, const Byte& rhs); friend bool operator&(const Byte& lhs, const Byte& rhs); */ }; class Char final : public Object{ / -m_data: char; type(void) const override -> Symbol; is_char(void) const override -> bool; lower(void) -> Char& upper(void) -> Char&; chr(u8) -> Char; ord(void) -> u8 issapce(void) -> bool ... / }; class Number final : public Object{ / +enum class Kind{INTEGER, FLOAT, COMPLEX}; -m_kind: Kind; -m_data: Any; // -- *builtin mathematical constants* -- :PI : Number :E : Number :PHI : NUmber :EPSILON : Number :IntMAX: Number :IntMIN: Number :FloatMIN: Number :FloatMAX: Number // -- *predicates* -- is_integer(void) const override; is_float(void) const override; is_complex(void) const override; is_number(void) const override;

// -- *common arithmetic functions* --// +, -, , /, % // -- common bitwise functions --// &, |, «, », ˆ, ~ // -- common logical functions --// &&, ||, ! // abs // -- rounding functions --// *round, ceil, truncate, floor,* // -- power & exponential & logarithmic functions --// *sqrt, hypot, pow, exp, exp2, expm1, log, log10, log2, log1p* // -- trigonometric functions -*-// sin, cos, tan, asin, acos, atan, atan2,

// -- *hyperpoblic functions* --// sinh, cosh, tanh, asinh, acosh, atanh, // lgamma, tgamma, // -- *error functions* --// erf, erfc, // -- *Complex specific functions* --// real, imag, arg, conj, polar, norm

*/

}; class String final : public Object, public Sequence { /* -m_data: Str; type(void) const override -> Symbol; is_string(void) const override -> bool; // -- *query functions* -- // len(void) const -> usize; // operator[](int) const -> char // operator[](int) -> char& // startswith(void) const -> bool // endswith(void) const -> bool // is_numeric(void) const -> bool // find(const Char& c) const -> i64 // contains(const Char& c) const -> bool // -- *transformation function* -- // upper, lower, ... // -- *modification functions* -- // split, join, replace, ... // -- *miscellaneous functions* -- // iter(void) const override -> Iterator; */ };

Compound Types

Tuple

code-block:: c++

class Tuple final : public Object, public Sequence{ /* -m_data: Vec<Self> +explicit Tuple() noexcept; +explicit Tuple(Vec<Self>&&) noexcept; +~Tuple(); type(void) const override -> Symbol; is_type(void) const override -> bool; +len(void) -> usize; +operator[](i64) const -> Self; +operator[](i64) -> Self& +iter(void) const override -> Iterator; ... */

};

Array

code-block:: c++

class Array final : public Object, public Sequence { /* -m_elementType: Symbol -m_capactiy: usize; -m_data: Vec<Self>; +explicit Array() noexcept; +explicit Array(usize size) noexcept; +~Array(); +type(void) const override -> Symbol; +is_tuple(void) const override -> bool; +iter(void) const override -> Self; ... */

};

BytesArray

code-block:: c++

class BytesArray final : public Object, public Sequence { /* -m_capacity: usize; -m_data: Vec<u8>; +explicit BytesArray() noexcept +explicit BytesArray(usize sz) noexcept; +type(void) const override -> Symbol; +is_bytearray(void)

const override -> bool; +iter(void) const override -> Iterator ... */

};

ArrayList

code-block:: c++

class ArrayList final : public Object, public Sequence{ /* -m_elementType: Symbol; -m_data: Vec<Self> +explicit ArrayList() noexcept; +explicit ArrayList(usize) noexcept; +type(void) const override -> Symbol; +is_arraylist(void) const override -> bool; +iter(void) const override -> Iterator, */

};

List

code-block:: c++

class List final : public Object, public Sequence { /* -m_elementType: Symbol; -m_data: ValueList +explicit List() noexcept; +explicit List(Self xs) noexcept; +type(void) const override -> Symbol; +is_list(void) const override -> bool; +iter(void) const override -> Iterator; ... */

};

HashSet

code-block:: c++

class SelfEqual; class SelfHash;

class HashSet final : public Object, public Mapping { /* -m_keyType: Symbol -m_data: Set<Self, SelfEqual, SelfHash> +explicit HashSet() noexcpet; +explicit HashSet(Self&& keys) noexcept; +type(void) const override -> Symbol; +is_hashset(void) const override -> bool; +iter(void) const override -> Iterator; ... */

};

HashMap

code-block:: c++

class SelfEqual; class SelfHash;

class HashMap final : public Object, public Mapping{
/* -m_keyType: Symbol -m_data: Dict<Self, Self, SelfEqual, SelfHash>; +explicit HashMap() noexcept; +explicit HashMap(Self&& keyvals) noexcept; +type(void) const override -> Symbol; +is_hashmap(void) const override -> bool; +iter(void) const override -> Iterator; ... */

};

Callable

code-block:: c++

class Callable final : public Object{ /* +enum class Kind{CFUN, LAMBDA, FUNCTION, MACRO}; -m_kind: Kind -m_ast: Ast; -m_env: Env; +explicit Callable(Str name, CFun) noexcept; +explicit Callable(Vec<Parameter>&& params, BlockStmtAst&& ast) noexcept +explicit Callable(Identifier&& name, Vec<Parameter>&& params, BlockStmtAst&& ast, bool macro=false) noexcept; +type(void) const override -> Symbol; +is_callable(void) const override -> bool; +is_lambda(void) const override -> bool; +is_function(void) const override -> bool; +is_cfunction(void) const override -> bool; +is_macro(void) const override -> bool; +operator()(Vec<Args>& args) -> Self; +min_argc(void) const -> i32; +max_argc(void) const -> i32; +argc(void) const -> i32; ... */

};

Iterable

code-block:: c++

class Iterable : public Object{ /* +virtual next(void) -> Self = 0; +virtual has_next(void) -> bool; +virtual map(Self&& fun) -> Iterator; +virtual map_while(Self&& fun) -> Iterator; +virtual zip<Iterator other) -> Iterator; +virtual zip(Vec<Iterator> others) -> Iterator; +virtual filter(Self&& predicate) -> Iterator; +virtual reduce(Self&& fun, Self init) -> Self; +virtual apply(Self&& fun) -> Self +virtual reverse(void) -> Iterator; +virtual flatten(void) -> Iterator; +virtual enumerate(void) -> Iterator; +virtual chain(Vec<Iterator>&& other) -> Iterator; +virtual take(void) -> Iterator; +virtual take_while(void) -> Iterator; +virtual to_string(void) -> String +virtual to_tuple(void) -> Tuple; +virtual to_array(void) -> Array; +virtual to_arraylist(void) -> ArrayList; +virtual to_list(void)

-> List; +virtual to_hashset(void) -> HashSet; +virtual to_hashmap(void) -> HashMap; */

};

StringIterator

code-block:: c++

class StringIterator final : public Iterable { /* -m_data: String; ... */

};

TupleLiteral

code-block:: c++

class TupleIterator final : public Iterable{ /* -m_data: Tuple; ... */

};

ArrayIterator

code-block:: c++

class ArrayIterator final : public Iterable{ /* -m_data: Array */

};

BytesArrayIterator

code-block:: c++

class BytesArrayIterator final : public Iterable{ /* -m_data: BytesArray */

};

ArrayListIterator

code-block:: c++

class ArrayListIterator final : public Iterable { /* -m_data: ArrayList */

}

ListIterator

code-block:: c++

```
class ListIterator final : public Iterable{ /* -m_data: List */
};
```

Range

code-block:: c++

```
class Range final : public Iterable{ /* -m_start: i64 -m_stop: i64 -m_step: i64 +explicit Range(i64 stop) noexcept; +explicit Range(i64 start, i64 stop) noexcept; +explicit Range(i64 start, i64 stop, i64 step) noexcept; ... */
};
```

Linspace

code-block:: c++

```
class Linspace final : public Iterable{ /* -m_start: f64 -m_stop: f64 -m_count: u64 +explicit Linspace(f64 start, f64 stop) noexcept; +explicit Linspace(f64 start, f64 stop, u64 count) noexcept; ... */
};
```

HashSetIterator

code-block:: c++

```
class HashSetIterator final : public Iterable{ /* -m_data: HashSet */
};
```

HashMapIterator

code-block:: c++

```
class HashMapIterator final : public Iterable{ /* -m_data: HashMap */
};
```

Structure

code-block:: c++

class Structure final : public Object{ /* m_ast: StructStmtAst +explicit Structure(StructStmtAst&& ast) noexcept; +type(void) const override -> Symbol; +is_newtype(void) const override -> bool; ... */

};

Trait

code-block:: c++

class Trait final : public Object{ /* -m_ast: TraitStmtAst; +explicit Trait(TraitStmtAst&& ast) noexcept; +type(void) const override -> Symbol; +is_trait(void) const override -> bool; ... */

};

Option

code-block:: c++

class Option final : public Object{ /* -m_data: Optional +explicit Option() noexecept; +explicit Option(Self&& self) noexcept; +type(void) const override -> Symbol; +is_optional(void) const override -> bool; +unwrap(void) -> Self; +value_or(Self&& self) -> Self; +is_none(void) -> bool; ... */

};

Result

code-block:: c++

class Result final : public Object{ /* +enum class Kind{OK, ERR}; +m_kind: Kind; +m_data: Any; +explicit Result(Self&& ok) noexcept; +explicit Result(Error&& err) noexcept; +type(void) const override -> Symbol; +is_ok(void) const override -> bool; +is_error(void) const override -> bool; +unwrap(void) -> Self; +is_error(void) -> bool; +error(void) -> Error; ... */

};

Syntax and AST

code-block:: c++

..comment:: ast-node-kind

BoolLiteral, ByteLiteral, CharLiteral, IntegerLiteral, FloatLiteral, StringLiteral, TupleLiteral, ArrayLiteral, HashSetLiteral, HashMapLiteral, Identifier, new, type, struct, fun, lambda, macro, let, var, @define, @operator, @overload, @property, @static, @testing, @test, @setup, @cleanup, @impl, import, from, alias, trait, implements, if, for, loop, else, elif, when, match, case break, continue, return, and, or, not Ok, Err, Some, None, delete, call

AstKind

code-block:: c++

```
#define ALIZE_AST_KINDS() ALIZE_DEF(BoolLITERAL, "BOOL_LITERAL") ALIZE_DEF(ByteLITERAL, "BYTE_LITERAL") ALIZE_DEF(CharLITERAL, "CHAR_LITERAL") ALIZE_DEF(IntegerLITERAL, "INTEGER_LITERAL") ALIZE_DEF(FloatLITERAL, "FLOAT_LITERAL") ALIZE_DEF(StringLITERAL, "STRING_LITERAL") ALIZE_DEF(TupleLITERAL, "TUPLE_LITERAL") ALIZE_DEF(ArrayLITERAL, "ARRAY_LITERAL") ALIZE_DEF(HashSetLITERAL, "HASHSET_LITERAL") ALIZE_DEF(HashMapLITERAL, "HASHMAP_LITERAL") ALIZE_DEF(UndefinedLITERAL, "UNDEFINED_LITERAL") ALIZE_DEF(IdentifierLITERAL, "IDENTIFIER_LITERAL") ALIZE_DEF(SelfLITERAL, "SELF_LITERAL") ALIZE_DEF(OkExprAST, "OK_EXPR") ALIZE_DEF(ErrExprAST, "ERR_EXPR") ALIZE_DEF(SomeExprAST, "SOME_EXPR") ALIZE_DEF(NoneExprAST, "NONE_EXPR") ALIZE_DEF(ExprStmtAST, "EXPR_STMT") ALIZE_DEF(UnaryExprAST, "UNARY_EXpR") ALIZE_DEF(BinaryExprAST, "BINARY_EXPR") ALIZE_DEF(TernaryExprAST, "TERNARY_EXPR") ALIZE_DEF(DeleteExprAST, "DELETE_EXPR") ALIZE_DEF(WhenExprAST, "WHEN_EXPR") ALIZE_DEF(CommaExprAST, "COMMA_EXPR") ALIZE_DEF(LambdaExprAST, "LAMBDA_EXPR") ALIZE_DEF(CallExprAST, "CALL_EXPR") ALIZE_DEF(CallMemberExprAST, "CALL_MEMBER_EXPR") ALIZE_DEF(GetIndexExprAST, "GET_INDEX_EXPR") ALIZE_DEF(SetIndexExprAST, "SET_INDEX_EXPR") ALIZE_DEF(GetMemberExprAST, "GET_MEMBER_EXPR") ALIZE_DEF(SetMemberExprAST, "SET_MEMBER_EXPR") ALIZE_DEF(AliasStmtAST, "ALIAS_EXPR_STMT") ALIZE_DEF(ReturnStmtAST, "RETURN_EXPR_STMT") ALIZE_DEF(NewStmtAST, "NEW_EXPR_STMT") ALIZE_DEF(TypeStmtAST, "TYPE_STMT") ALIZE_DEF(StructStmtAST, "STRUCT_STMT") ALIZE_DEF(FunStmtAST, "FUN_STMT") ALIZE_DEF(MacroStmtAST, "MACRO_STMT") ALIZE_DEF(LetStmtAST, "LET_STMT") ALIZE_DEF(VarStmtAST, "VAR_STMT") ALIZE_DEF(AtDefineStmtAST, "AT_DEFINE_STMT") ALIZE_DEF(AtOperatorStmtAST, "AT_OPERATOR_STMT") ALIZE_DEF(AtPropertiesStmtAST, "AT_PROPERTIES_STMT")
```

ALIZE_DEF(AtStaticStmtAST, "AT_STATIC_STMT") ALIZE_DEF(AtTestingStmtAST, "AT_TESTING_STMT") ALIZE_DEF(AtTestStmtAST, "AT_TEST_STMT") ALIZE_DEF(AtSetupStmtAST, "AT_SETUP_STMT") ALIZE_DEF(AtCleanupStmtAST, "AT_CLEANUP_STMT") ALIZE_DEF(ImportStmtAST, "IMPORt_STMT") ALIZE_DEF(TraitStmtAST, "TRAIT_STMT") ALIZE_DEF(ImplementsStmtAST, "IMPLEMENTS_STMT") ALIZE_DEF(AtImplStmtAST, "AT_IMPL_STMT") ALIZE_DEF(IfStmtAST, "IF_STMT") ALIZE_DEF(ForStmtAST, "FOR_STMT") ALIZE_DEF(LoopStmtAST, "LOOP_STMT") ALIZE_DEF(CaseStmtAST, "CASE_STMT") ALIZE_DEF(MatchStmtAST, "MATCH_STMT") ALIZE_DEF(BlockStmtAST, "BLOCK_STMT") ALIZE_DEF(BreakStmtAST, "BREAK_STMT") ALIZE_DEF(ContinueStmtAST, "CONTINUE_STMT")

AstBase

code-block:: c++

```
class AstBase{ /* +AstBase(kind: AstKind) noexcept; +virtual ~AstBase() = default; #m_kind: AstKind #m_token: Token // the "trigger" token #m_lineStart: std::string::iterator #m_lineEnd: std::string::iterator #m_lineno: usize #m_start: usize #m_end: usize +virutal repr(void) -> std::string = 0 */

};
```

ExprAstBase

code-block:: c++

```
class ExprAstBase : public AstBase{ /* +virtual ~ExprAstBase() = default; +virtual eval(Alize& alize) -> Self = 0; */

};
```

StmtAstBase

code-block:: c++

```
class StmtAstBase : public AstBase{ /* +virtual ~StmtAstBase() = default; +virtual execute(Alize& alize, Self& result) -> void = 0; */

};
```

BoolExprAst

code-block:: c++

```
class BoolExprAst final : public ExprAstBase { /*      bool
    value; */

};
```

ByteExprAst

code-block:: c++

```
class ByteExprAst final : public ExprAstBase{ /* u8 value;
    */

};
```

CharExprAst

code-block:: c++

```
class CharExprAst final : public ExprAstBase{ /*      char
    value; */

};
```

IntegerExprAst

code-block:: c++

```
class IntegerExprAst final : public ExprAstBase{ /*     i64
    value; */

};
```

FloatExprAst

code-block:: c++

```
class FloatExprAst final : public ExprAstBase{ /* f64 value;
    */

};
```

StringExprAst

code-block:: c++

```
class StringExprAst final : public ExprAstBase{ /*      Str
    value; */

};
```

TupleExprAst

code-block:: c++

```
class TupleExprAst final : public ExprAstBase{ /* u8 value; Vec<Ast> data; // vector-of-expressions */

};
```

ArrayExprAst

code-block:: c++

```
class ArrayExprAst final : public ExprAstBase{ /* Symbol itemType; // elements type Vec<Ast> data; // vector-of-expressions */

};
```

HashSetExprAst

code-block:: c++

```
class HashSetExprAst final : public ExprAstBase{ /* Symbol keyType; Set<Ast, AstEqual, AstHash> data; */

};
```

HashMapExprAst

code-block:: c++

```
class HashMapExprAst final : public ExprAstBase{ /* Symbol keyType; // Bool, Integer, String, Dict<Ast, Ast, AstEqual, AstHash> data; */

};
```

UndefinedExprAst

code-block:: c++

```
class UndefinedExprAst final : public ExprAstBase{ /* Symbol value; */

};
```

IdentifierExprAst

code-block:: c++

class IdentifierExprAst final : public ExprAstBase{ /* Symbol value; bool typeBound; // true in let name=expr, false in var name=expr bool conceal; // marked with @conceal to make it private bool constant; // true in @define(name, expr), false otherwise */

};

OkExprAst

code-block:: c++

class OkExprAst final : public ExprAstBase{ /* Ast expr; // Ok(expr) */

};

ErrExprAst

code-block:: c++

class ErrExprAst final : public ExprAstBase{ /* Ast expr; // Err(expr) | Err("name", expr) */

};

SomeExprAst

code-block:: c++

class SomeExprAst final : public ExprAstBase{ /* Ast expr; // Some(expr) */

};

NoneExprAst

code-block:: c++

class NoneExprAst final : public ExprAstBase{ /* Symbol value; // None */

};

UnaryExprAst

code-block:: c++

class UnaryExprAst final : public ExprAstBase{ /* op rhsExpr Str op; Ast rhsExpr; */

};

BinaryExprAst

code-block:: c++

```
class BinaryExprAst final : public ExprAstBase{ /*      lhsExpr op rhsExpr Ast lhsExpr; Str op; Ast rhsExpr; */

};
```

TernaryExprAst

code-block:: c++

```
class TernaryExprAst final : public ExprAstBase{ /* Ast[3] exprs; */

};
```

DeleteExprAst

code-block:: c++

```
class DeleteExprAst final : public ExprAstBase{ /*
    Vec<Identifier> names; // delete name1, name2, ..., nameN */

};
```

WhenExprAst

code-block:: c++

```
class WhenExprAst final : public ExprAstBase{ /* when(test){}{} TernaryExprAst ast; */

};
```

CommaExprStmtAst

code-block:: c++

```
class CommaExprStmtAst final : public ExprAstBase{
    /* id1, id2, ..., idN = expression | expr1, expr2, ..., exprN
    Vec<Ast> exprs; // let x, y = 1, 2 or let x, _ = (1, 2) */

};
```

LambdaExprAst

code-block:: c++

```
class LambdaExprAst final : public ExprAstBase{ /*
    Vec<Parameter> params; Vec<Ast> body; Str docstr; */
};
```

CallExprAst

code-block:: c++

```
class CallExprAst final : public ExprAstBase{ /*        Ast
    callableExpr; Vec<Ast> args; */
};
```

CallMemberExprAst

code-block:: c++

```
class CallMemberExprAst final : public ExprAstBase{ /*
    obj.member(args) | Type:member(args) | Structure:member(args)
    Ast objExprAst; // instance | Type | Structure Identifier
    member; Vec<Ast> args; */
};
```

GetIndexExprAst

code-block:: c++

```
class GetIndexExprAst final : public ExprAstBase{ /*
    obj[idx] BinaryExprAst ast; // instance */
};
```

SetIndexExprAst

code-block:: c++

```
class SetIndexExprAst final : public ExprAstBase{ /*
    obj[idx] = expr TupleExprAst ast; */
};
```

GetMemberExprAst

code-block:: c++

```
class GetMemberExprAst final : public ExprAstBase{ /*
    obj.member BinaryExprAst ast; */
};
```

SetMemberExprAst

code-block:: c++

```
class SetMemberExprAst final : public ExprAstBase{ /*
    obj.member = expr TernaryExprAst ast; */

};
```

ExprStmtAst

code-block:: c++

```
class ExprExprAst final : public ExprAstBase{ /* Ast expr;
    */

};
```

AliasStmtAst

code-block:: c++

```
class AliasStmtAst final : public StmtAstBase{ /* alias Int
    = Integer; Identifier neoId; Identifier oldId; */

};
```

ReturnExprStmtAst

code-block:: c++

```
class ReturnExprStmtAst final : public StmtAstBase{ /*
    return | return expr; Ast expr; */

};
```

NewExprStmtAst

code-block:: c++

```
class NewExprStmtAst final : public StmtAstBase{ /* new
    ArrayList(); Ast expr; */

};
```

TypeStmtAst

code-block:: c++

```
class TypeStmtAst final : public StmtAstBase{ /* type Person = {
        @properties{...} @static{ ... } @operator +(other){ ... }
        @impl ___str___(){ ... } fun is_contractor(){...}
```

}; Identifier name; Ast initfn; // @initialize() Set<Property, PropertyEqual, PropertyHash> properties; Dict<Identifier, FunStmtAst> methods; Dict<Identifier, FunStmtAst> operators; Dict<Identifier, SharedPtr<TraitStmtAst» traitsImpl; Dict<Identifier, Ast> staticItems; Str docstr; */

};

StructStmtAst

code-block:: c++

class StructStmtAst final : public StmtAstBase{ /* Identifier name; Ast initfn; // @initialize(...){ ... } Set<Property, PropertyEqual, PropertyHash> properties; Dict<Identifier, FunStmtAst> methods; Dict<Identifier, FunStmtAst> operators; Dict<Identifier, SharedPtr<TraitStmtAst» traitsImpl; Dict<Identifier, Ast> staticItems; Str docstr; */

};

FunStmtAst

code-block:: c++

class FunStmtAst final : public StmtAstBase{ /* // (1) fun name(){ body } // (2) fun name(x, y){ body } // (3) fun name(x, y, ?opt1, ?opt2=2){ body } // (4) fun name(x, y, key1=val1, key2=val){ body } // (5) fun name(x, y, ?opt, key=val){ body } // (6) fun name(x, y, key=val, ?opt=val){ body }

Identifier name; Vec<Parameter> params; BlockStmtAst body; Str docstr; */

};

MacroStmtAst

code-block:: c++

class MacroStmtAst final : public StmtAstBase{ /* macro name(params){["docstr" || """docstrs"""], body } Identifier name; Vec<Parameter> params; BlockStmtAst body; Str docstr; */

};

LetStmtAst

code-block:: c++

class LetStmtAst final : public StmtAstBase{ /* Identifier name; Ast expr; */

};

VarStmtAst

code-block:: c++

class VarStmtAst final : public StmtAstBase{ /* Identifier name; Ast expr; */

};

AtDefineStmtAst

code-block:: c++

class AtDefineStmtAst final : public StmtAstBase{ /* // @define(name, expr) // @define(name, expr, docstr) Identifier name, Ast expr; Str docstr; */

};

AtOperatorStmtAst

code-block:: c++

class AtOperatorStmtAst final : public StmtAstBase{ /* @operator +(other){ [docstr], body } Str op; Vec<Parameter> params; BlockStmtAst body; Str docstr; */

};

AtPropertiesStmtAst

code-block:: c++

class AtPropertiesStmtAst final : public StmtAstBase{ /* @properties{ field1[= expr], ... } Set<Property, PropertyEqual, PropertyHash> properties; */

};

AtStaticStmtAst

code-block:: c++

class AtStaticStmtAst final : public StmtAstBase{ /* Dict<Identifier, Ast> staticFields; */

};

AtTestingStmtAst

code-block:: c++

    class AtTestingStmtAst final : public StmtAstBase{ /* Ast setupAst; Ast cleanupAst; Dict<Identifier, Ast> testsAst; */

    };

AtTestStmtAst

code-block:: c++

    class AtTestStmtAst final : public StmtAstBase{ /* @test name { body } | @test name{ @skip, body } BlockStmtAst body; bool failed; bool skipped; */

    };

AtSetupStmtAst

code-block:: c++

    class AtSetupStmtAst final : public StmtAstBase{ /* @setup{ body } Dict<Identifier, Ast> info; */

    };

AtCleanupStmtAst

code-block:: c++

    class AtCleanupStmtAst final : public StmtAstBase{ /* @cleanup{ body } BlockStmtAst body; */

    };

ImportStmtAst

code-block:: c++

    class ImportStmtAst final : public StmtAstBase{ /* Identifier modulename; // import modulename Set<Identifier> symbols; // import { symbols } from modulename */

    };

TraitStmtAst

code-block:: c++

```
class TraitStmtAst final : public StmtAstBase{ /* trait Widget {
        @properties{ ... } @fun draw(color, label, parent)

        fun display(){ println(format("I am {self.name} whídegt
            type"))

        }

    } Identifier name; Set<Property, PropertyEqual, Property-
    Hash> properties; Set<Signature> signatures; // fun show()
    Dict<Identifier, FunStmtAst> funcs; // fun prinln(...){ ... }
    Str docstr; */

};
```

ImplementsStmtAst

code-block:: c++

```
class ImplementsStmtAst final : public StmtAstBase{ /*
    Set<Identifier> traitsId; */

};
```

AtImplStmtAst

code-block:: c++

```
class AtImplStmtAst final : public StmtAstBase{ /* Identi-
    fier name; Vec<Parameter> params; BlockStmtAst body; */

};
```

IfStmtAst

code-block:: c++

```
class IfStmtAst final : public StmtAstBase{ /* Ast testExpr;
    BlockStmtAst okBody; Vec<std::pair<Ast, BlockStmtAst» elifs;
    BlockStmtAst elseBody; */

};
```

ForStmtAst

code-block:: c++

```
class ForStmtAst final : public StmtAstBase{ /* // for(x:
    range(1, 10)){ body } Identifier elemId; Ast iteratorExpr;
    BlockStmtAst body; u8 level; bool continueJumped; bool
    breakJumped; */
```

};

LoopStmtAst

code-block:: c++

class LoopStmtAst final : public StmtAstBase{ /* loop{ body } Vec<Ast> body; u8 level; bool continueJumped; bool breakJumped; */

};

MatchStmtAst

code-block:: c++

class MatchStmtAst final : public StmtAstBase{ /* match expr { ... } Ast expr; Vec<CaseStmtAst> cases; */

};

CaseStmtAst

code-block:: c++

class CaseStmtAst final : public StmtAstBase{ /* case(clause){ body } Ast matchClause; BlockStmtAst body; */

};

BlockStmtAst

code-block:: c++

class BlockStmtAst final : public StmtAstBase{ /* @scope{ body } Vec<Ast> body; bool allowedContinue; bool allowedBreak; */

};

BreakStmtAst

code-block:: c++

class StructStmtAst final : public StmtAstBase{ /* u8 level; */

};

ContinueStmtAst

code-block:: c++

```
class StructStmtAst final : public StmtAstBase{ /* u8 level; */

};
```

Precedence and Associativity

Core Data Structures

code-block:: c++

```
class Arg final : Object{ /* -m_param: Parameter; +Arg(Identifier, Self, Parameter::Kind); ... */

};

class IdentifierEqual final{ /* +operator()(const Identifier& lhs, const Identifier& rhs) -> bool */

};

class IdentifierHash final{ /* +operator()(const Identifier& ident) -> size_t; */

};

class Parameter final{ /* +enum class Kind{POSITIONAL, OPTIONAL, KEYVAL}; +kind: Kind; +ident: Identifier; +value: Self; */

};

class ParameterEqual final{ /* +operator()(const Parameter& lhs, const Parameter& rhs) -> bool; */

};

class ParameterHash final{ /* +operator()(const Parameter& param) -> size_t; */

};

class Signature final{ /* +ident: Identifier; +params: Set<Parameter, ParameterEqual, ParameterHash>; */

};

class SignatureEqual final{ /* +operator()(const Signature& lhs, const Signature& rhs) -> bool; */
```

};

class SignatureHash final{ /* +operator()(const Signature& signature) -> size_t; */

};

class Property final{ /* +ident: Identifier; +value: Self; */

};

class PropertyEqual final{ /* +operator()(const Property& lhs, const Property& rhs) -> bool; */

};

class PropertyHash final{ /* +operator()(const Property& rhs) -> size_t; */

};

class Method final{ /* +enum Kind{INSTANCE_METHOD, STATIC_METHOD}; +ident: Identifier; +fun: Self; */

};

class MethodEqual final{ /* +operator()(const Method& lhs, const Method& rhs) -> bool; */

};

class MethodHash final{ /* +operator()(const Method& method) -> size_t; */

};

class Symbol final{ /* -m_data: Str; */

};

class SymbolEqual final{ /* operator()(const Symbol& lhs, const Symbol& rhs) ->bool; */

};

class SymbolHash final{ /* operator()(const Symbol& self) -> size_t; */

};

class Error final{ /* +enum Kind{ Default, NotImplemented, Syntax, Value, Type, Runtime, System, }; -m_prefix: Str; -m_kind: Symbol; -m_reason: Self; -m_msg: Str; -m_env: Env; +show(void) const -> Str; +prefix(void) const -> const Str&; +prefix(void) -> Str&; +kind(void) const -> Symbol&; +kind(void) -> Symbol& */

};

class Env final{ /* -m_bindings: Dict<Identifier, Self, IdentifierEqual, IdentifierHash>; -m_parent: Env*; +add(const Identifier& key, const Self& val) -> void; +get(cnst Identifier& key) -> Self; +update(const Identifier& key, const Self& val) -> Self; +hasKey(const Identifier& key) -> bool; +contains(const Identifier& key) -> bool; +str(void) const -> Str; friend std::ostream& operator«(std::ostream&, const Env&); */

};

class Module final{ /* -m_name: Symbol; -m_path: fs::path; -m_env: Env; */

};

class ModuleEqual final { /* operator()(const Module& lhs, const Module& rhs) const -> bool */

};

class ModuleHash final { /* operator()(const Module& self) const -> size_t; */

};

class SelfEqual final { /* operator()(const Self& lhs, const Self& rhs) -> bool */

};

class SelfHash final { /* operator()(const Self& self) -> size_t; */

};

class Matcher final { /* -m_expr: Ast; +explicit Matcher(Ast& expr) noexcept; +match(const Matcher& matcher) const -> bool; -match(OkExprAst& ast) -> bool; -match(ErrExprAst& ast) -> bool; -match(IntegerLiteral& ast) -> bool; -match(FloatLiteral& ast) -> bool; -match(StringLiteral& ast) -> bool; -match(SomeExprAst& ast) -> bool; -match(NoneExprAst& ast) -> bool; -match(UndefinedExprAst& ast) -> bool; -match(BoolLiteral& ast) -> bool; -match(TupleExprAst& ast) -> bool; -match(ArrayExprAst& ast) -> bool; -match(ArrayListExprAst& ast) -> bool; -match(ListExprAst& ast) -> bool; -match(HashSetExprAst& ast) -> bool; -match(HashMapExprAst& ast) -> bool; -match(TypeStmtAst& ast) -> bool; -match(StructStmtAst& ast) -> bool; -match(Identifier& ast) -> bool; */

};

class Collectable{ /* +virutal len(void) const -> usize = 0; #virtual add(Self&& self) -> void = 0; #virtual remove(const Self& self) -> void = 0; #virtual contains(const Self& self) const -> bool = 0; */

};

class Sequence : public Collectable { /* #+virtual slice(void) const -> Self; #+virtual get(u64 idx) const -> Self; #+virtual set(u64 idx) -> Self; */

};

class Mapping : public Collectable { /* +virtual keys(void) const -> Tuple; +virtual values(void) const -> Tuple; +virtual getEntry(const Self& key) -> Array; +virtual getEntry(const Self& key, const Self& defVal) -> Array; +virtual setEntry(Self&& key, Self&& value) -> void +virtual popEntry(void) -> Array; +virtual entries(void) -> Array */

};

Self share(void); Self share(const Undefined& undefined); Self share(u8 b); Self share(bool flag); Self share(char c); Self share(i64 num); Self share(f64 num); Self share(Str str, bool symbol=false); // kind="tuple" | "array" | "arraylist" | "list" | "bytearray" Self share(const Vec<Self>& vec, const Str kind="tuple"); Self share(const Set<Self, SelfHash, SelfEqual>& hset); Self share(const Dict<Self, Self, SelfHash, SelfEqual>& hmap); Self share(Tuple&& tuple); Self share(Array&& array); Self share(String&& string); Self share(ArrayList&& arraylist); Self share(List&& xs); Self share(HashSet&& hset); Self share(HashMap&& hmap); Self share(Structure&& ty); // kind="ok", "none", "some" Self share(Self&& self, const Str& kind="ok"); Self share(Error&& err);

// undefined, "[]literals" Ast make_ast(Token, Str kind="..."); // kind="tuple" | "array" | "arraylist" | "list" | "bytesarray" Ast make_ast(Token, Vec<Ast>&& exprs, Str, String kind=""); // other-literals Ast make_ast(Token, Set<Ast, AstHash, AstEqual>); // hashset Ast make_ast(Token, Dict<Ast, Ast, AstHash, AstEqual>); // hashmap Ast make_ast(Token, bool flag); // bool Ast make_ast(Token, u8 b); // byte Ast make_ast(Token, Vec<Parameter>, BlockStmtAst&&); // lambda // kind="macro" | "fun" Ast make_ast(Token, Identifier, Vec<Parameter>, BlockStmtAst&&, Str kind); Ast make_ast(Token, Identifier, Set<Property>, Set<Signature>); // trait Ast make_ast(Token, Set<Identifier>); // ImplementsStmtAst Ast make_ast(Token, Identifier, ImplementsStmtAst* ast, Dict<Identifier, Ast>&&, // properties FunStmtAst&&, // initfn Dict<Identifier, FunStmtAst>&&,

// methods Dict<Identifier, FunStmtAst>&&, // operators Dict<Identifier, FunStmtAst>&&, // impls Dict<Identifier, Ast>&& // staticFields); // struct Ast make_ast(Token, Str, Ast&&); // unary // kind="arithmetic-binary-ops" | "Get[Member|Index]" Ast make_ast(Token, Ast&&, Str, Ast&&, Str kind); // binary // kind="Set[Member|Index] | when" Ast make_ast(Token, Ast[3], Str kind); // ternary Ast make_ast(Token, Identifier, Ast&&, BlockStmtAst&&); // for Ast make_ast(Token, BlockStmtAst&& ast); // loop Ast make_ast(Token, Ast&&, Vec<Ast>&&, Ast&&); // if Ast make_ast(Token, Ast&& ast); // break, continue Ast make_ast(Token, Ast&& ast, Ast&& expr); // return Ast make_ast(Token, Identifier, Vec<Ast>&&); // call Ast make_ast(Token, Identifier, Str op, Identifier, Vec<Ast>); //membercall // kind="@[operator|impl|static|hide|properties|testing|test|cleanup]" // "@[setup|skip|initialize]" Ast make_ast(Token, Identifier, Ast&& ast, Str kind);

Tokenizer

code-block:: c++

struct Location{ /* rowStart: Str::iterator; rowEnd: Str::iterator; row: i64; col: i64; */

};

struct Token{ /* kind: TokenKind; lexeme: Str lineno: i32; loc: Location; */

};

class Tokenizer{ /* -m_beg: Str::iterator; -m_end: Str::iterator; -m_ptr: Str::iterator; -m_char: char; -m_loc: Location; -m_lineno: i32; ... */

};

..token-kinds:

```
"Invalid", "Eof", ".", ":", "::", "[", "]", "(", ")", "{", "}",
"+", "-", "*", "/", "%", "==", "!=", "<=", ">=", "<", ">", "<<",
">>", "|", "&", "~","and", "or", "not", "undefined", "self",
"None", "Some", "Ok", "Err", "true", "false", "struct", "macro",
"fun", "lambda", "trait", "@fun", "@initialize", "@properties",
"@impl", "@testing", "@test", "@setup", "@cleanup", "@static",
"@skip", "@main", "for", "loop", "continue", "break", "return",
"if", "elif", "else", "new", "type", "(*", "*)", "'''", "@hide",
```

Parser

code-block:: c++

class Parser final{ /* +m_enum Kind{FILE, STRING}; -m_kind: Kind; -m_src: Any -m_tokenizer: Tokenizer; ... */

};

Alize Interpreter

code-block:: c++

class Alize{ /* +struct Version{ ... }; ::version: Version; ::license: std::string; ::authors: Set<std::string>; ::prelude : Dict<std::string, Self, std::hash<Str>, std::equal<Str»; ::modules : Set<Module, ModuleHash, ModuleEqual>; ::builtinDocs: Dict<Str, Str, std::hash<Str>, std::equal<Str»; ::userDocs: Dict<Symbol, Str, SymbolHash, SymbolEqual> ::builtinFuncs: Set<Symbol, SymbolHash, SymbolEqual>; ::builtinConstants: Dict<Symbol, SymbolHash, SymbolEqual> ::builtinErrors: Set<Symbol, SymbolHash, SymbolEqual>; ::is_keyword(std::string) -> bool ::is_reserved_word(std::string) -> bool ::is_valid_identifier_start(char) -> bool ::is_valid_identifier_char(char) -> bool ::is_whitespace(char) -> bool ::is_comment_start(const Str&) -> bool ::is_comment_end(const Str&) -> bool; ::is_numstr(const std::string&) -> bool ::report_error(std::string::itertor ptr, const Error&) -> void

-m_runtime: Dict<Identifier, Self> -m_importedModules: Set<Module, ModuleHash, ModuleEqual>

+eval(void) -> Self ::repl(opts: Vec<std::string>) -> void ::run(fname: std::string, args: Vec<std::string>) -> void ::run(opts: Vec<std::string>, fname: std::string, args: Vec<std::string>) -> void //-eval(Env& env) -> Self //-execute(Env& env) -> void -import_module(Module& module, const Env&) -> Env -load_binary_module(const const fs::path& dynlib_path, const Str& name) -> Env; -eval(const [...]ExprAst& ast) -> Self -execute(const [...]StmtAst& ast) -> void */

};

Builtin Functions and Constants

..comment: builtin modules

traits, filesystem, datetime, io, json, yaml, toml, xml,

..comment: builtin constants

true, false, undefined

..comment: builtin functions

Bool, Char, Integer, Float, Complex, String, Tuple, Array, ArrayList, List, HashSet, HashMap, StringIterator, TupleIterator, ArrayIterator, ArrayListIterator, ListIterator, HashSetIterator, HashMapIterator, typeof, isinstance, range, linspace, iterator, format, println, print, eprintln, eprint, panic, hash, next, eval, assert, assertEqual, assertTrue, assertTrue, assertOk, assertErr, assertResult, assertSome, assertNone, assertOptional, assertLess, assertLessEqual, assertGreater, assertGreaterEqual, assertInstance, assertTypeof, assertZero, assertUndefined,

..comment: builtin operators +, -, , /,, <, <=, >, >=, ==, !=, «, », |, &, ^, ~,

..comment: builtin implementable functions

___hash___, ___str___, ___repr___, ___len___, ___call___ ___get_prop___, ___set_prop___, ___delete_prop___